Oasis Core 프로세스 맵

소개

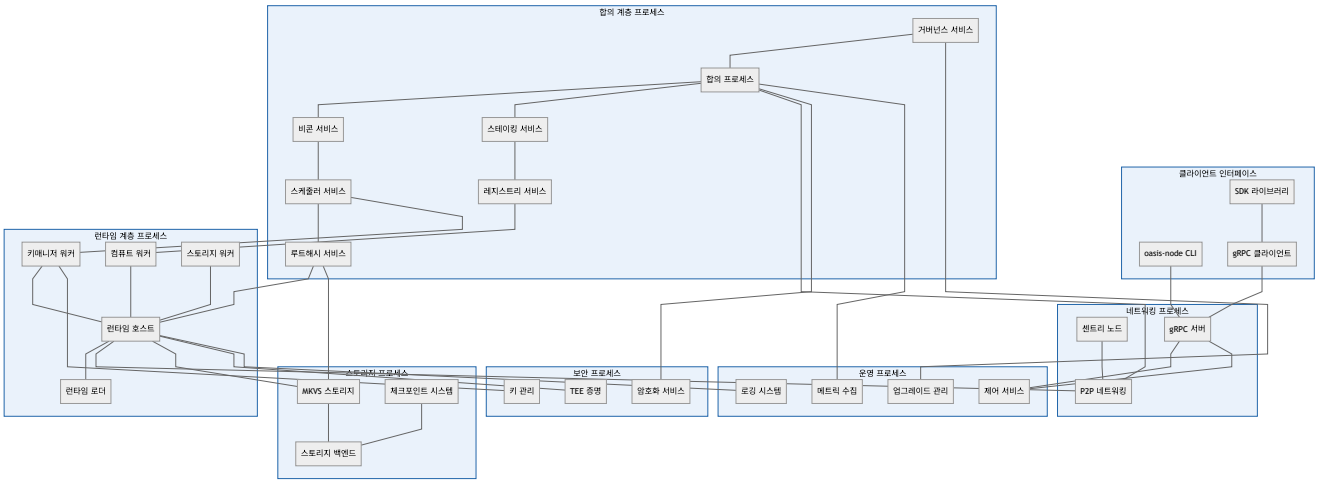
Oasis Core는 다양한 독립적인 프로세스들이 협력하여 전체 블록체인 네트워크를 구성하는 분산 시스템입니다. 이 문서는 Oasis Core의 모든 프로세스를 체계적으로 분류하고, 각 프로세스의 역할과 상호 관계를 설명합니다.

프로세스 분류 체계

Oasis Core의 프로세스들은 다음과 같이 6개의 주요 카테고리로 분류됩니다:

1. **합의 프로세스**: 블록체인 합의와 상태 관리
2. **런타임 프로세스**: 애플리케이션 실행과 상태 처리
3. **네트워킹 프로세스**: P2P 통신과 네트워크 관리
4. **스토리지 프로세스**: 데이터 저장과 검증
5. **보안 프로세스**: 키 관리와 암호화
6. **운영 프로세스**: 모니터링과 관리

전체 프로세스 관계도


합의 계층 프로세스
거버넌스 서비스
합의 프로세스
비콘 서비스
스테이킹 서비스
스케줄러 서비스
레지스트리 서비스
루트해시 서비스
런타임 계층 프로세스
키매니저 워커
컴퓨트 워커
스토리지 워커
런타임 호스트
런타임 로더
클라이언트 인터페이스
SDK 라이브러리
oasis-node CLI
gRPC 클라이언트
네트워킹 프로세스
센트리 노드
gRPC 서버
P2P 네트워킹
스토리지 프로세스
MKVS 스토리지
체크포인트 시스템
스토리지 백엔드
보안 프로세스
키 관리
TEE 증명
암호화 서비스
운영 프로세스
로깅 시스템
메트릭 수집
업그레이드 관리
제어 서비스

프로세스별 상세 분류

1. 합의 계층 프로세스

합의 계층은 블록체인의 기본적인 합의와 상태 관리를 담당하는 프로세스들로 구성됩니다.

1.1 합의 프로세스 (Consensus Process)

위치: `go/consensus/`

역할:

- CometBFT 기반 비잔틴 장애 허용 합의 알고리즘 실행
- 블록 생성과 검증
- 네트워크 상태 일관성 유지

주요 기능:

- 트랜잭션 순서 결정
- 블록 제안과 투표
- 합의 상태 관리
- 밸리데이터 세트 관리

의존성:

- 비콘 서비스 (랜덤성)
- 스테이킹 서비스 (밸리데이터 정보)
- P2P 네트워킹 (노드 간 통신)

1.2 비콘 서비스 (Beacon Service)

위치: `go/beacon/`

역할:

- 검증 가능한 랜덤성 제공
- 에포크 기반 시간 관리

- 네트워크 스케줄링 지원

주요 기능:

- VRF(Verifiable Random Function) 기반 랜덤 비콘 생성
- 에포크 전환 관리
- 시간 기반 이벤트 트리거

의존성:

- 합의 프로세스 (블록 높이 정보)
- 스케줄러 서비스 (에포크 정보 제공)

1.3 스테이킹 서비스 (Staking Service)

위치: `go/staking/`

역할:

- PoS(Proof of Stake) 메커니즘 구현
- 경제적 보안 제공
- 밸리데이터 인센티브 관리

주요 기능:

- 스테이킹과 언스테이킹 처리
- 보상 계산과 분배
- 슬래싱 처리
- 위임 관리

의존성:

- 합의 프로세스 (밸리데이터 정보)
- 레지스트리 서비스 (노드 정보)

1.4 레지스트리 서비스 (Registry Service)

위치: `go/registry/`

역할:

- 네트워크 엔티티 등록 관리
- 노드와 런타임 메타데이터 관리
- 공개키 배포

주요 기능:

- 엔티티 등록과 검증
- 노드 등록과 업데이트
- 런타임 등록과 관리
- 메타데이터 저장

의존성:

- 합의 프로세스 (상태 저장)
- 스테이킹 서비스 (엔티티 검증)

1.5 스케줄러 서비스 (Scheduler Service)

위치: `go/scheduler/`

역할:

- 런타임 위원회 구성
- 노드 역할 할당
- 실행 스케줄링

주요 기능:

- 위원회 선택 알고리즘

- 역할 기반 노드 할당
- 스케줄 최적화
- 위원회 교체 관리

의존성:

- 비콘 서비스 (랜덤성)
- 레지스트리 서비스 (노드 정보)
- 스테이킹 서비스 (스테이크 정보)

1.6 거버넌스 서비스 (Governance Service)

위치: `go/governance/`

역할:

- 네트워크 거버넌스 관리
- 제안과 투표 처리
- 네트워크 업그레이드 조정

주요 기능:

- 제안 생성과 검증
- 투표 메커니즘
- 가중치 계산
- 업그레이드 실행

의존성:

- 합의 프로세스 (제안 처리)
- 스테이킹 서비스 (투표 가중치)

1.7 루트해시 서비스 (RootHash Service)

위치: `go/roothash/`

역할:

- 런타임 상태 커밋 처리
- 상태 루트 검증
- 디스크레판시 탐지

주요 기능:

- 런타임 상태 커밋
- 상태 루트 합의
- 실행 결과 검증
- 디스크레판시 해결

의존성:

- 합의 프로세스 (상태 저장)
- 런타임 호스트 (상태 정보)

2. 런타임 계층 프로세스

런타임 계층은 실제 애플리케이션 로직이 실행되는 프로세스들로 구성됩니다.

2.1 런타임 호스트 (Runtime Host)

위치: `go/runtime/host/`

역할:

- 런타임 실행 환경 관리
- 런타임과 합의 계층 간 통신
- 실행 결과 처리

주요 기능:

- 런타임 프로세스 관리
- 런타임 호스트 프로토콜 처리
- 상태 동기화
- 결과 검증

의존성:

- 런타임 로더 (런타임 바이너리)
- 루트해시 서비스 (상태 커밋)
- MKVS 스토리지 (상태 저장)

2.2 런타임 로더 (Runtime Loader)

위치: `runtime-loader/`

역할:

- 런타임 바이너리 로딩
- 런타임 검증
- 실행 환경 초기화

주요 기능:

- 런타임 번들 처리
- 바이너리 검증
- 메타데이터 파싱
- 실행 환경 설정

의존성:

- 런타임 호스트 (로딩 요청)
- TEE 증명 (보안 검증)

2.3 컴퓨트 워커 (Compute Worker)

위치: `go/worker/compute/`

역할:

- 일반적인 트랜잭션 실행
- 상태 변경 처리
- 실행 결과 생성

주요 기능:

- 트랜잭션 배치 처리
- 상태 읽기/쓰기
- 실행 증명 생성
- 결과 커밋

의존성:

- 런타임 호스트 (실행 환경)
- 스토리지 워커 (상태 데이터)
- 레지스트리 서비스 (위원회 정보)

2.4 키매니저 워커 (KeyManager Worker)

위치: `go/worker/keymanager/`

역할:

- 암호화 키 관리
- 기밀 컴퓨팅 지원
- 키 분배와 회전

주요 기능:

- 분산 키 생성
- 키 에스크로
- 키 회전 처리
- 기밀 데이터 처리

의존성:

- 런타임 호스트 (실행 환경)
- 키 관리 서비스 (키 저장)
- TEE 증명 (보안 환경)

2.5 스토리지 워커 (Storage Worker)

위치: `go/worker/storage/`

역할:

- 런타임 상태 저장
- 상태 동기화
- 체크포인트 생성

주요 기능:

- 상태 데이터 관리
- 동기화 처리
- 체크포인트 생성
- 상태 검증

의존성:

- MKVS 스토리지 (데이터 저장)
- 체크포인트 시스템 (스냅샷)

- P2P 네트워킹 (동기화)

3. 네트워킹 프로세스

네트워킹 프로세스는 노드 간 통신과 네트워크 관리를 담당합니다.

3.1 P2P 네트워킹 (P2P Networking)

위치: `go/p2p/`

역할:

- 노드 간 분산 통신
- 피어 발견과 관리
- 메시지 라우팅

주요 기능:

- libp2p 기반 네트워킹
- 피어 발견 프로토콜
- 메시지 전파
- 연결 관리

의존성:

- 센트리 노드 (보안 연결)
- 레지스트리 서비스 (노드 정보)

3.2 센트리 노드 (Sentry Node)

위치: `go/worker/sentry/`

역할:

- 밸리데이터 보호
- 네트워크 접근 제어

- DDoS 방어

주요 기능:

- 트래픽 필터링
- 연결 제한
- 밸리데이터 익명화
- 공격 탐지

의존성:

- P2P 네트워킹 (연결 관리)
- 레지스트리 서비스 (노드 인증)

3.3 gRPC 서버 (gRPC Server)

위치: `go/consensus/api/grpc.go`

역할:

- 클라이언트 API 제공
- 외부 인터페이스 관리
- 요청 처리

주요 기능:

- API 엔드포인트 제공
- 인증과 권한 관리
- 요청 라우팅
- 응답 처리

의존성:

- 제어 서비스 (내부 통신)

- 합의 프로세스 (상태 조회)

4. 스토리지 프로세스

스토리지 프로세스는 데이터 저장과 검증을 담당합니다.

4.1 MKVS 스토리지 (MKVS Storage)

위치: `go/storage/mkvs/`

역할:

- Merkle Tree 기반 검증 가능한 스토리지
- 상태 무결성 보장
- 효율적인 상태 증명

주요 기능:

- Merkle Tree 구조 관리
- 상태 증명 생성
- 노드 압축
- 버전 관리

의존성:

- 스토리지 백엔드 (데이터 저장)
- 체크포인트 시스템 (스냅샷)

4.2 체크포인트 시스템 (Checkpoint System)

위치: `go/storage/mkvs/checkpoint/`

역할:

- 상태 스냅샷 생성
- 빠른 동기화 지원

- 상태 복구

주요 기능:

- 체크포인트 생성
- 압축과 배포
- 검증과 복구
- 증분 업데이트

의존성:

- MKVS 스토리지 (상태 데이터)
- 스토리지 백엔드 (저장)

4.3 스토리지 백엔드 (Storage Backend)

위치: `go/storage/`

역할:

- 실제 데이터 저장
- 다양한 데이터베이스 지원
- 성능 최적화

주요 기능:

- BadgerDB, LevelDB 지원
- 트랜잭션 처리
- 압축과 정리
- 백업과 복구

의존성:

- 파일 시스템 (데이터 저장)

5. 보안 프로세스

보안 프로세스는 키 관리와 암호화를 담당합니다.

5.1 키 관리 (Key Management)

위치: `go/keymanager/`

역할:

- 암호화 키 생성과 관리
- 키 분배와 회전
- 보안 정책 적용

주요 기능:

- 키 생성과 저장
- 키 에스크로
- 접근 제어
- 감사 로그

의존성:

- TEE 증명 (보안 환경)
- 암호화 서비스 (암호화 연산)

5.2 TEE 증명 (TEE Attestation)

위치: `go/ias/`

역할:

- 신뢰 실행 환경 검증
- 원격 증명 처리
- 하드웨어 보안 확인

주요 기능:

- SGX 증명 검증
- 증명서 관리
- 신뢰 체인 구축
- 보안 정책 적용

의존성:

- Intel IAS (증명 서비스)
- 암호화 서비스 (서명 검증)

5.3 암호화 서비스 (Crypto Service)

위치: `go/common/crypto/`

역할:

- 암호화 연산 제공
- 디지털 서명 처리
- 해시 함수 제공

주요 기능:

- 서명 생성과 검증
- 해시 계산
- 암호화/복호화
- 키 유도

의존성:

- 하드웨어 보안 모듈 (선택적)

6. 운영 프로세스

운영 프로세스는 시스템 모니터링과 관리를 담당합니다.

6.1 메트릭 수집 (Metrics Collection)

위치: 각 모듈의 `metrics/` 디렉토리

역할:

- 성능 메트릭 수집
- 상태 모니터링
- 알림 생성

주요 기능:

- Prometheus 메트릭 제공
- 성능 지표 수집
- 상태 체크
- 알림 트리거

의존성:

- 모든 프로세스 (메트릭 소스)

6.2 로깅 시스템 (Logging System)

위치: `go/common/logging/`

역할:

- 구조화된 로그 관리
- 로그 수집과 저장
- 디버깅 지원

주요 기능:

- 구조화된 로깅

- 로그 레벨 관리
- 로그 라우팅
- 로그 보관

의존성:

- 모든 프로세스 (로그 소스)

6.3 제어 서비스 (Control Service)

위치: `go/control/`

역할:

- 노드 제어와 관리
- 내부 API 제공
- 상태 관리

주요 기능:

- 노드 상태 관리
- 내부 API 제공
- 설정 관리
- 서비스 제어

의존성:

- gRPC 서버 (API 제공)
- 모든 서비스 (제어 대상)

6.4 업그레이드 관리 (Upgrade Management)

위치: `go/upgrade/`

역할:

- 네트워크 업그레이드 조정
- 버전 관리
- 호환성 확인

주요 기능:

- 업그레이드 스케줄링
- 버전 호환성 체크
- 업그레이드 실행
- 롤백 지원

의존성:

- 거버넌스 서비스 (업그레이드 승인)
- 모든 서비스 (업그레이드 대상)

프로세스 간 의존성과 통신

의존성 매트릭스

| 프로세스 | 합의 | 런타임 | 네트워킹 | 스토리지 | 보안 | 운영 |

|---|---|---|---|---|---|---|

| **합의** | ● | → | → | → | → | ← |

| **런타임** | ← | ● | → | → | → | ← |

| **네트워킹** | ← | ← | ● | - | → | ← |

| **스토리지** | ← | ← | - | ● | → | ← |

| **보안** | ← | ← | ← | ← | ● | ← |

| **운영** | → | → | → | → | → | ● |

범례:

- ●: 내부 의존성
- →: 서비스 제공
- ←: 서비스 사용
- -: 직접적 의존성 없음

통신 방식

1. 동기 통신

- **gRPC**: 클라이언트-서버 간 API 호출
- **함수 호출**: 같은 프로세스 내 모듈 간 통신
- **IPC**: 프로세스 간 직접 통신

2. 비동기 통신

- **이벤트 버스**: 상태 변경 알림
- **메시지 큐**: 작업 큐잉과 처리
- **P2P 메시징**: 노드 간 분산 통신

3. 데이터 공유

- **공유 스토리지**: MKVS를 통한 상태 공유
- **캐시**: 자주 사용되는 데이터 캐싱
- **설정**: 공통 설정 정보 공유

이러한 프로세스 맵을 통해 Oasis Core의 복잡한 시스템 구조를 체계적으로 이해하고, 각 프로세스의 역할과 상호작용을 파악할 수 있습니다.

마지막 업데이트: 2024년 1월

작성자: Oasis Core 문서화 팀

검토자: 시스템 아키텍처 팀